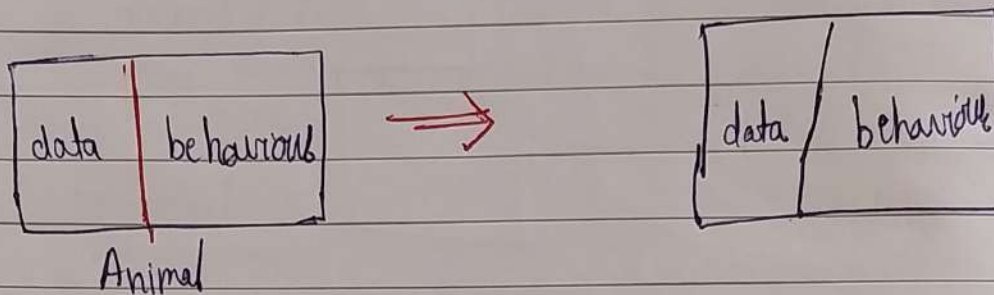
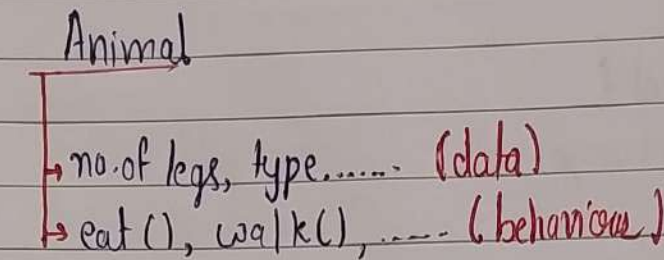


Encapsulation & Inheritance

* Encapsulation

- ↳ to form a capsule
- a real life data is encapsulated



```
class Animal {  
    int noOfLegs;  
    int id;  
    String type;  
    }  
    void eat() {  
        //code  
    }  
    void run() {  
        //code  
    }  
}
```

→ data

— behaviour

→ Encapsulation

- both data & behaviours should be together (encapsulated) within an object
- we should not provide unrestricted access of data.

eg: Bank account
 - balance (data)
 - bankName, ifsc,

→ deposit (behaviour)
 → withdraw

```
class BankAccount {
    double balance;
    String bankName;
    String IFSC;
    ...
}
```

BankAccount ba = new BankAccount();
 ba.balance = 1,00,000; (X)
 (not possible in real life)

* Access Modifiers

→ who have access to (variable, method, constructor, class)

We have 4 access Modifiers

public
 default
 protected
 private

4 keywords

```
class BankAccount {  
    private double balance;  
    ...  
}
```

```
}
```

```
BankAccount ba = new BankAccount();  
ba.balance; // not accessible
```

* Private

```
class BA {  
    private double balance;
```

```
    void withdraw() {  
        balance--; // code  
    }
```

```
}
```

cannot be access outside
this class

* default (by default)

→ Anyone can access belonging to same package;

```
public class Demo {  
    void main() {  
        //  
    }  
}
```

a package group similar
classes / interfaces together.

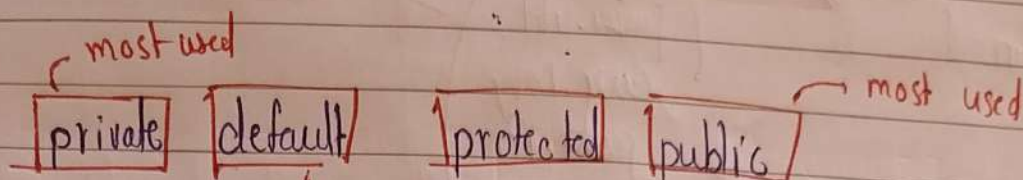
```
class Animal {  
    //  
}
```


* protected

→ can access within same package + inherited class (child class)

* public

→ anyone can access



Strictness: ←

cannot use
private &
protected in
class

(not allowed in java) →
but we can
use them
in internal
class

```
public class Demo {  
    void main() {  
    }  
}  
→ class Student {  
    {  
    }
```

because if we did make class private then we can never access that class & also in case of protected we can't even inherit whole class.

```
class BankAccount {  
    private double balance;  
    void deposit (int amount) {  
        this.balance += amount;  
    }  
    void withdraw (int amount) {  
        this.balance -= amount;  
    }  
}
```

```
BankAccount ba  
= new BankAccount();  
  
// ba.balance = 100;  
// can't possible  
ba.deposit(800);  
// possible  
ba.withdraw(300);  
// possible
```

getter & setter : we'll see them in code.

* packages

user
defined

in built

(java.util
java.lang
java.io
...)

libraries

```
import java.util. ArrayList;
```

```
public class DemoMain {
```

```
void main () {
```

```
}
```

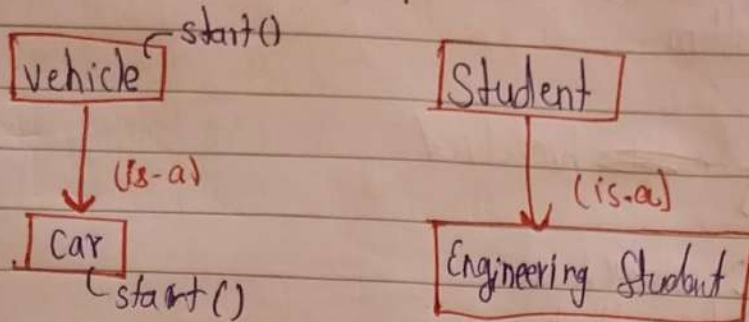
```
}
```

Demo.java

Byte code

Demo.class

* Inheritance → provides code reusability.



```

class Vehicle {
    void start() {
        cout << "Vehicle started";
    }
}

class Car extends Vehicle {
    void setGear() {
        // code
    }
}
  
```

Car c = new Car();
 c.start();
 c.setGear();

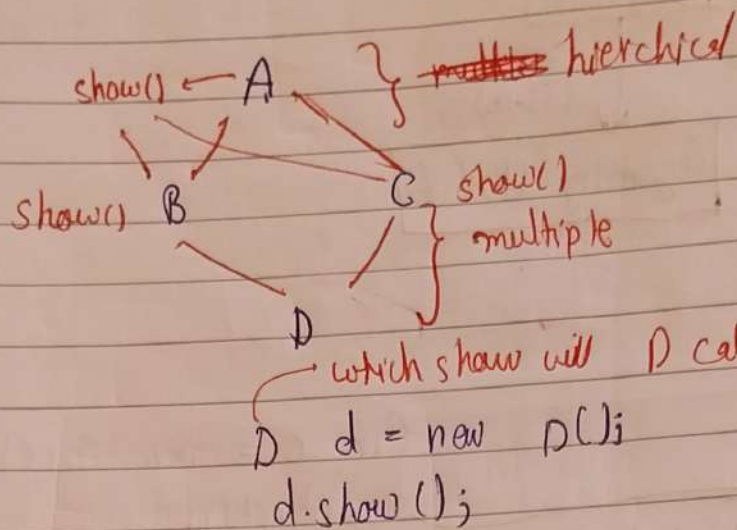
↑ is a

keyword to use inheritance

Inheritance

- code reusability
- supports polymorphism

Q Why Multiple Inheritance is not supported in java
 → Due to Diamond problem



but to support multiple inheritance, we use interfaces which we'll see later

* Super keyword

→ store reference of parent object

this → store ref of current object

* Uses of super

① to access parent class variable

class A {

int x = 4;

}

class B extends A {

out(super.x);

}

② I can call parent class method

class A {

void greet()

}

class B extends A {

void sayHello() {

super.greet();

}

}

③ Call parent class constructor:

3] call parent class constructor

```
class A {  
    A() {  
    }  
}
```

```
class B extends A {  
    B() {  
        Super();  
    }  
}
```

Call this